■ AceAI

AI-Powered Mock Interview Preparation Platform

1. Project Overview

AceAI is a Generative AI–powered mock interview platform built for a hackathon. Its goal is to help job seekers **simulate real-world interviews**, **practise answering questions out loud**, and receive **personalised AI-driven feedback** — all without needing a human interviewer. The platform covers the full interview lifecycle: from resume analysis and question generation, through to real-time speech recording and detailed post-session coaching.

Core Value Proposition
Upload your resume & job description → Get 10 AI-tailored interview questions → Answer via microphone or text → Receive detailed AI coaching feedback instantly.

2. System Architecture

The application follows a clean **client–server architecture** with two independently running services that communicate over HTTP REST.

Layer	Technology	Port
Frontend	Next.js 15 (App Router, Turbopack) · React 19 · TailwindCSS 3	3000
Backend API	Flask 3.1 · Python 3 · Flask-CORS · PyPDF2	5000
AI / LLM	Google Gemini gemini-2.5-flash via google-generativeai SDK	Cloud

3. Full Application Flow

Step 1 — Landing Page

The user lands on the AceAI homepage, which displays a hero section with a headline ('Master Your Dream Job Interviews With AI') and a call-to-action button. No login or registration is required.

Step 2 — Resume Upload Modal

Clicking 'Start your interview prep here' opens a modal form where the user provides: (a) their **Resume as a PDF** (drag-and-drop or file browse), (b) the **Job Description** as free text, and (c) an optional **Knowledge Domain** (e.g., 'AI, ML, Web Dev').

Step 3 — AI Question Generation

On clicking 'Launch', the frontend sends a **POST /upload** request with the resume PDF and job description. The backend extracts text from the PDF using PyPDF2, then constructs a Gemini prompt. Gemini returns a JSON object containing **10 tailored interview questions** matching both the candidate's background and the role requirements.

Step 4 — Live Interview Screen

The modal closes and the interview screen activates. The user's **webcam and microphone turn on automatically**. Questions appear one at a time. The user clicks 'Start Answer', speaks their response (captured by the browser's **Web Speech API / SpeechRecognition**), then clicks 'Stop Recording'. If speech recognition is unavailable they can type manually.

Step 5 — Per-Answer AI Analysis

After each answer, a **POST /analyze** request is sent with the question and the transcribed answer. The backend computes: **filler word percentage** (um, uh, like, basically…) and **repeated words count** locally, then asks Gemini AI for: **constructive feedback** (2–3 sentences), **relevance** (High / Medium / Low), and **sentiment** (Positive / Neutral / Negative).

Step 6 — Full Feedback Report

Once every question is answered, the interview ends and the webcam stops. A **complete report card** is displayed, showing for every question: the question, the user's answer, AI coaching feedback, filler %, relevance, repeated-word count, and sentiment. The user can then return to the home screen and start a new session.

4. AI & LLM Integration Details

Both AI tasks use Google's **gemini-2.5-flash** model via the official Python SDK (*google-generativeai*). The API key is loaded from an environment variable (*GEMINI_API_KEY*).

Task	Input to Gemini	Output from Gemini
Interview Question Generation	Extracted resume text + job description	`JSON: {"questions": ["Q1", "Q2", …]}`
Answer Analysis	Interview question + candidate's spoken answer	`JSON: {"feedback": "…", "relevance": "High", "sentiment": "Positive"}`

5. Backend API Reference

Method	Endpoint	Description	Key Fields
GET	/	Health check — returns service status and available endpoints	—
POST	/upload	Upload resume PDF + job description; returns 10 AI-generated questions	file (PDF), job_description, knowledge_domain
POST	/analyze	Analyse a candidate answer; returns AI feedback + local stats	question, response

6. Full Tech Stack

Category	Technology
Frontend Framework	Next.js 15 with App Router and Turbopack
UI Library	React 19
Styling	TailwindCSS 3
State Management	Redux 5 + React-Redux 9
PDF Handling	pdf-lib, pdfjs-dist (frontend); PyPDF2 (backend)
Backend Framework	Flask 3.1
AI Provider	Google Gemini (gemini-2.5-flash) via google-generativeai
Speech Input	Browser Web Speech API (SpeechRecognition)
Camera / Video	Browser MediaDevices API (getUserMedia)
Cross-Origin	Flask-CORS
Environment Config	.env.local (frontend) · .env (backend)

7. Key File Reference

File	Role
UI-main/src/app/page.js	Root page — controls landing vs. interview state

components/UserInputs.js	Resume upload modal form + /upload API call
components/askQuestions.js	Interview screen — webcam, mic controls, speech-to-text, feedback display
llm-api/app/routes.py	Flask route definitions for /upload and /analyze
llm-api/app/services.py	All Gemini AI logic — question generation and answer analysis functions
llm-api/app/config.py	Loads GEMINI_API_KEY from environment variables
llm-api/app/__init__.py	Flask app factory — registers CORS, blueprint, and error handlers

8. Potential Features to Add

■ High Impact

- Overall score (0–100) synthesising relevance, sentiment and filler percentage
- Timer per question (e.g. 2-minute countdown) to simulate real interview pressure
- Resume strength analyser — score the resume before the interview begins
- Text-to-Speech — browser SpeechSynthesis reads questions aloud to the user
- Category-based questions — tag each question as Behavioural / Technical / Situational

■ Analytics & Progress

- Progress bar across questions (e.g. 'Q 3 of 10')
- Radar / spider chart visualising skills: confidence, clarity, relevance
- Session history persisted in localStorage or a lightweight database

■ Richer AI Capabilities

- Follow-up questions — AI probes deeper when an answer is weak
- Ideal answer comparison — show a model answer alongside AI feedback
- Voice tone / confidence analysis from audio pitch and pacing data

■ UX Improvements

- Dark mode support
- Smooth animated transitions between questions
- Fully mobile-responsive layout
- Loading skeleton while Gemini generates questions